

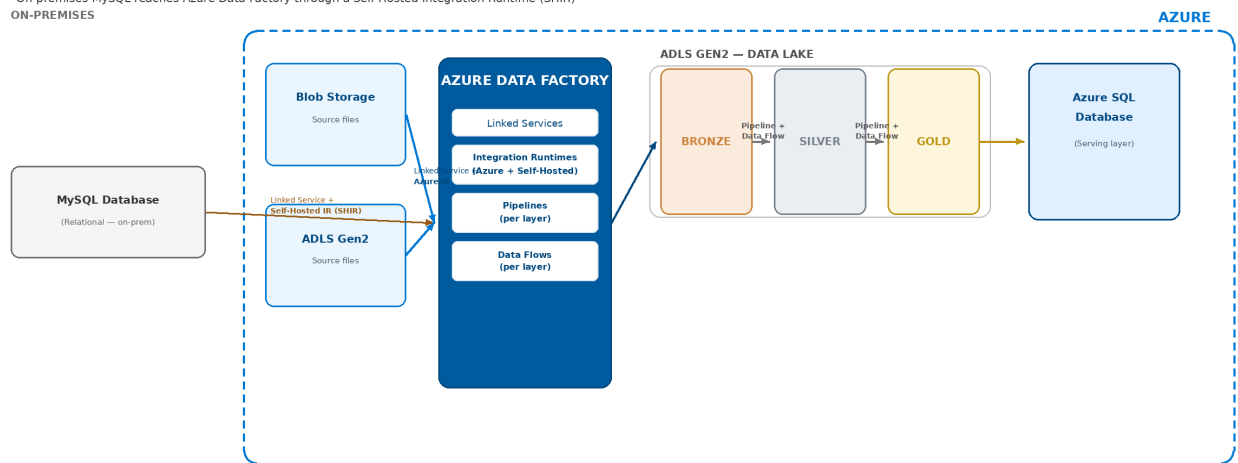
Apollo

Azure End-to-End Data Engineering Project

Batch ETL Pipeline built on the Medallion Architecture
(Bronze → Silver → Gold) using Azure Data Factory

Apollo — Azure End-to-End Data Engineering Architecture

On-premises MySQL reaches Azure Data Factory through a Self-Hosted Integration Runtime (SHIR)
ON-PREMISES



All Azure resources organised under one Resource Group (apollo_RS) | On-prem MySQL connects only via the Self-Hosted Integration Runtime

BLOB STORAGE

ADLS GEN2

AZURE DATA
FACTORY

AZURE SQL
DATABASE

MYSQL (ON-PREM)

A hands-on Azure project covering ingestion of healthcare-domain data — Patient, Provider, PCP, Encounter, Diagnosis, Pregnancy, and Location records — from an on-premises MySQL database plus Blob Storage and ADLS Gen2, orchestrated with Azure Data Factory and transformed through Bronze, Silver, and Gold, all the way to a relational serving layer in Azure SQL Database.

Blob Storage and ADLS Gen2 are native Azure services, connected to Data Factory through standard Linked Services and the default Azure Integration Runtime. MySQL, however, is an **on-premises database** — it connects through a **Self-Hosted Integration Runtime (SHIR)**, which lets Data Factory securely reach systems that aren't hosted in Azure. From there, ADF orchestrates a dedicated pipeline and data flow for each medallion layer — landing raw data in Bronze, cleaning and

standardizing it in Silver, and joining/aggregating the final business-ready dataset in Gold, which is also written out to Azure SQL Database.

What is Medallion Architecture?

Medallion Architecture is a data design pattern used in modern data lakes to incrementally improve the structure and quality of data as it flows through three layers — **Bronze, Silver, and Gold**. Instead of transforming raw data into a final report in one giant step, each layer has a single clear responsibility. This makes pipelines easier to debug, reprocess, and trust.

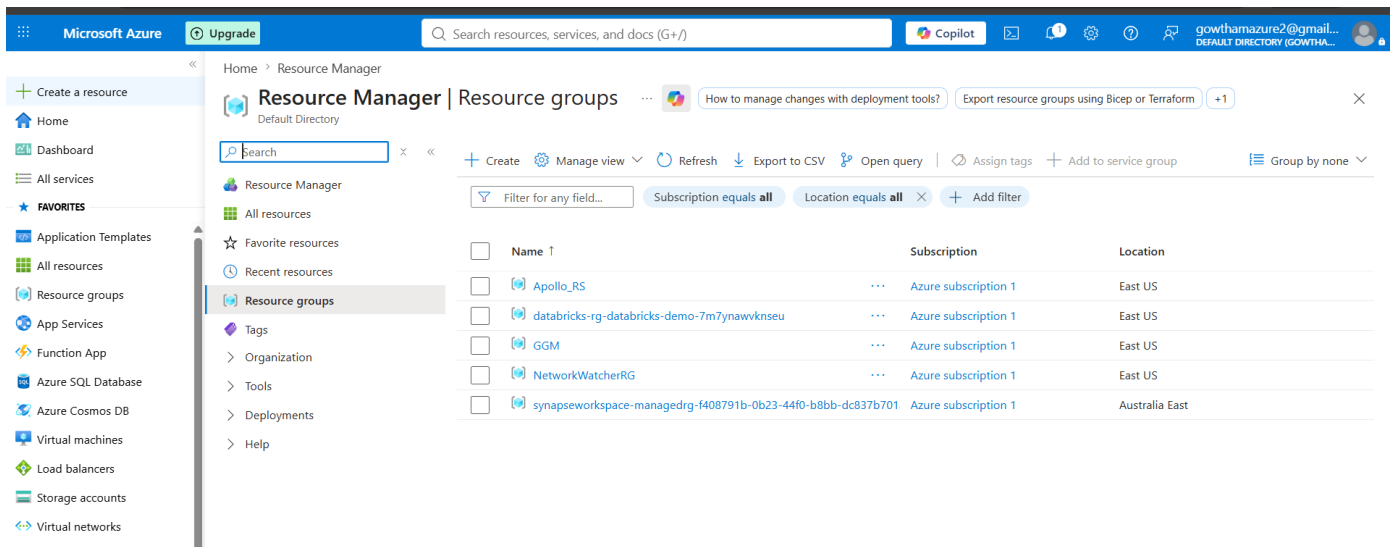


Bronze	Raw landing zone. Files from Blob Storage and ADLS Gen2 are copied in as-is, preserving full history so any step downstream can be reprocessed from scratch if logic changes.
Silver	Cleansed and conformed zone. A dedicated data flow removes bad/duplicate records, standardizes column types, and validates the data — turning raw files into a trusted, query-ready shape.
Gold	Business/aggregated zone. The final data flow joins and aggregates the Silver data into the output that's consumed downstream — written to both the Gold layer and Azure SQL Database.

Why it matters: each layer is independently testable, storage is cheap so keeping raw history is fine, and if business logic changes you only have to re-run Silver/Gold — not re-pull data from the original source systems.

Azure Resource Group

Step 1 — Provisioning the Resource Group



The screenshot shows the Azure Resource Manager console. The left sidebar contains navigation links for 'Create a resource', 'Home', 'Dashboard', 'All services', and 'FAVORITES'. Under 'FAVORITES', there are links for 'Application Templates', 'All resources', 'Resource groups', 'App Services', 'Function App', 'Azure SQL Database', 'Azure Cosmos DB', 'Virtual machines', 'Load balancers', 'Storage accounts', and 'Virtual networks'. The main area is titled 'Resource Manager | Resource groups' and shows a list of resource groups. The list has columns for 'Name', 'Subscription', and 'Location'. The resource groups listed are 'Apollo_RS', 'databricks-rg-databricks-demo-7m7ynawvkneue', 'GGM', 'NetworkWatcherRG', and 'synapseworkspace-managedrg-f408791b-0b23-44f0-b8bb-dc837b701'. All are under 'Azure subscription 1' and in 'East US' or 'Australia East' locations.

Name	Subscription	Location
Apollo_RS	Azure subscription 1	East US
databricks-rg-databricks-demo-7m7ynawvkneue	Azure subscription 1	East US
GGM	Azure subscription 1	East US
NetworkWatcherRG	Azure subscription 1	East US
synapseworkspace-managedrg-f408791b-0b23-44f0-b8bb-dc837b701	Azure subscription 1	Australia East

Every component of this project — storage, orchestration, and the database — lives inside a single Azure Resource Group. Grouping resources this way keeps billing, access control, and lifecycle management simple: the entire environment can be monitored, scaled, or deleted as one unit.

How it's used: Used as the logical container for all other services in this build.

- Resource group: **apollo_RS**

All Provisioned Services

Step 2 — All Services Deployed

Home > Resource Manager | Resource groups

Apollo_RS
Resource group

Search

+ Create Manage view Delete resource group Refresh Export to CSV Open query Assign tags Group by none

Overview

Activity log

Access control (IAM)

Tags

Resource visualizer

Events

Settings

Cost Management

Monitoring

Automation

Help

Essentials

Resources Recommendations

Filter for any field... Type equals all Location equals all Add filter

Name ↑	Type	Location
Apollo_db (apolloproject/Apollo_db)	SQL database	Australia East
apolloadf1232	Data factory (V2)	East US
apolloadlstorage	Storage account	East US
apolloblobstorage1234	Storage account	East US
apolloproject	SQL server	Australia East

Here's the full stack provisioned inside the resource group: storage accounts for the data lake, Azure Data Factory for orchestration, and Azure SQL Database for serving — all deployed together to keep networking simple and latency low.

How it's used: Forms the backbone infrastructure that the rest of the pipeline runs on.

Azure Blob Storage

Step 3 — Source Data in Blob Storage

Home > Resource Manager | Resource groups > Apollo_RS > apolloblobstorage1234

apolloblobstorage1234 | Storage browser

Storage account

Blob containers > apolloblobstorage1234

Authentication method: Access key (Switch to Microsoft Entra user account)

Add filter

Search blobs by prefix (case-sensitive)

Only show active blobs

Showing all 7 items

Name	Last modified	Access tier	Blob type	Size	Lease state
Encounter_Table.xlsx	7/24/2026, 10:16:39 PM	Hot (Inferred)	Block blob	5.2 MiB	Available
Encounter_dx_Table.csv	7/24/2026, 10:16:39 PM	Hot (Inferred)	Block blob	6.45 MiB	Available
Location_Table.csv	7/24/2026, 10:16:36 PM	Hot (Inferred)	Block blob	234 B	Available
OnpremData.csv	7/25/2026, 11:35:54 AM	Hot (Inferred)	Block blob	1.37 MiB	Available
PCP_Table.csv	7/24/2026, 10:16:37 PM	Hot (Inferred)	Block blob	468.85 KiB	Available
Pregnancy_Diagnosis.csv	7/24/2026, 10:16:36 PM	Hot (Inferred)	Block blob	15.35 KiB	Available
Provider.json	7/24/2026, 10:16:37 PM	Hot (Inferred)	Block blob	6.36 KiB	Available

One of three source systems for this project: Azure Blob Storage, holding raw files that feed the pipeline — a common landing spot for files dropped by upstream systems or vendors.

How it's used: Read by an ADF pipeline as a file-based source dataset.

ADLS Gen2

Step 4 — Source Data in ADLS Gen2

The screenshot shows the Azure Storage browser interface for a storage account named 'apolloadlstorage'. The left sidebar contains navigation links for 'Create a resource', 'Home', 'Dashboard', 'All services', and 'FAVORITES' (Application Templates, All resources, Resource groups, App Services, Function App, Azure SQL Database, Azure Cosmos DB, Virtual machines, Load balancers, Storage accounts, Virtual networks). The main pane shows the breadcrumb path 'Home > Resource Manager | Resource groups > Apollo_RS > apolloadlstorage'. Below this, the title is 'apolloadlstorage | Storage browser' with a 'Storage account' subtitle. A toolbar includes 'Add Directory', 'Upload', 'Refresh', 'Delete', 'Copy', 'Paste', 'Rename', 'Acquire lease', 'Break lease', and 'Edit columns'. The current view is 'Blob containers > apollo > medallion'. The authentication method is 'Access key (Switch to Microsoft Entra user account)'. A search bar is present with the placeholder 'Search blobs by prefix (case-sensitive)' and a filter 'Only show active objects'. Below the search bar, it says 'Showing all 3 items'. A table lists the items:

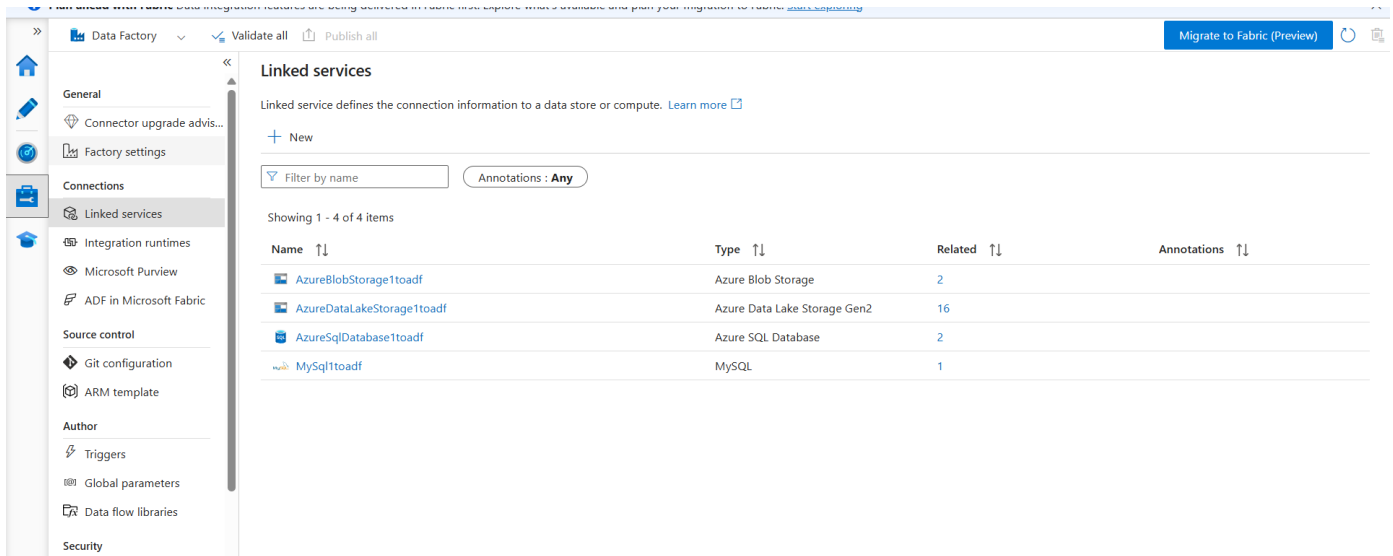
	Name	Last modified	Access tier	Blob type	Size	Lease state
	[-]					...
	bronze	7/24/2026, 10:17:41 PM				...
	gold	7/24/2026, 10:17:55 PM				...
	silver	7/24/2026, 10:17:48 PM				...

A second source: Azure Data Lake Storage Gen2, which adds a hierarchical namespace on top of Blob Storage for better performance and folder-level access control on big-data workloads. This same storage account also hosts the Bronze/Silver/Gold layers themselves.

How it's used: Acts both as a source system and as the landing zone for all three medallion layers.

ADF Linked Services

Step 5 — Connecting Every Source (Linked Services)



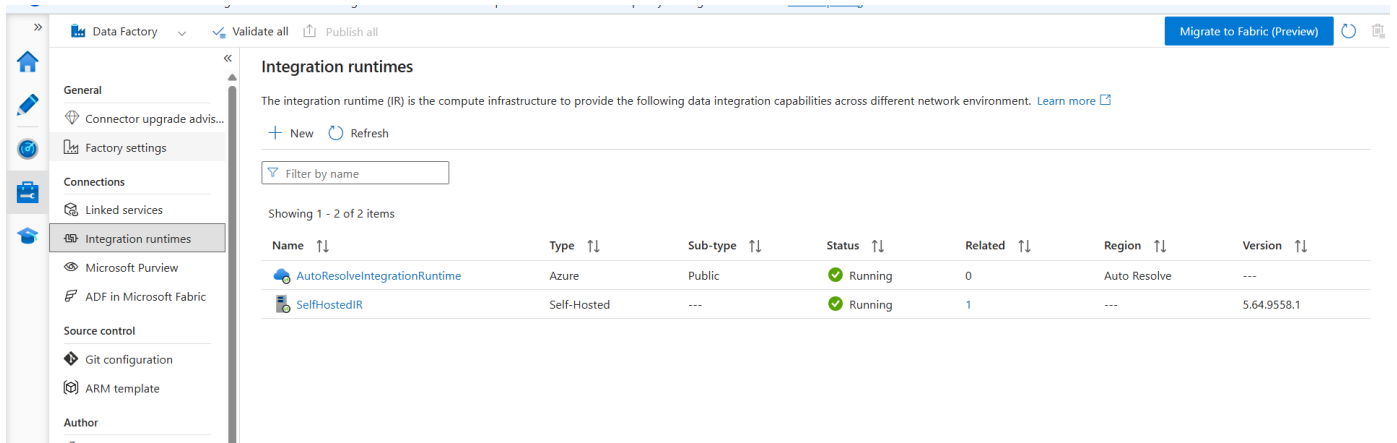
The third source is a relational MySQL database running **on-premises / outside Azure** — a realistic multi-source scenario. Blob Storage, ADLS Gen2, MySQL, and Azure SQL Database are each registered in Data Factory as a Linked Service — the reusable connection definition ADF uses to reach that system.

How it's used: Linked Services are the foundation every dataset and pipeline activity is built on top of.

- 4 linked services: Blob Storage, ADLS Gen2, MySQL (on-prem), Azure SQL Database

Integration Runtime

Step 6 — Configuring the Integration Runtime



The screenshot shows the 'Integration runtimes' page in the Azure Data Factory portal. The left-hand navigation pane includes options like 'General', 'Connections', and 'Integration runtimes'. The main content area is titled 'Integration runtimes' and contains a table listing the configured runtimes. The table has columns for Name, Type, Sub-type, Status, Related, Region, and Version. Two runtimes are listed: 'AutoResolveIntegrationRuntime' (Azure, Public, Running) and 'SelfHostedIR' (Self-Hosted, Running).

Name	Type	Sub-type	Status	Related	Region	Version
AutoResolveIntegrationRuntime	Azure	Public	Running	0	Auto Resolve	---
SelfHostedIR	Self-Hosted	---	Running	1	---	5.64.9558.1

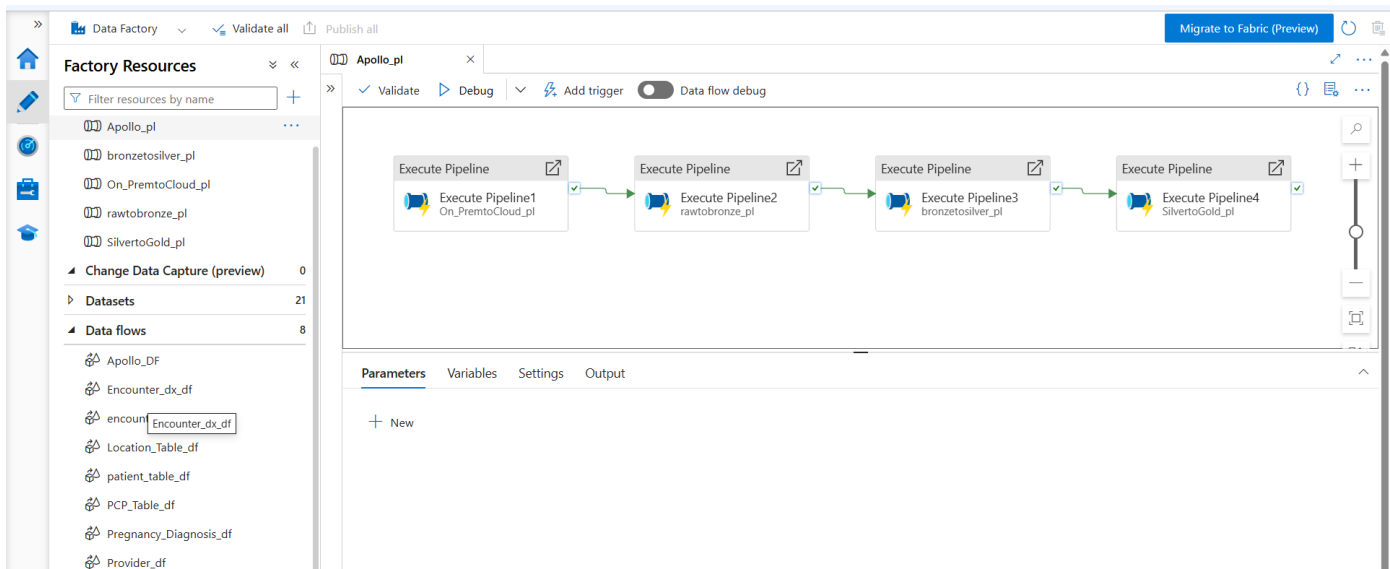
Blob Storage and ADLS Gen2 use the standard **Azure Integration Runtime** since they're native Azure services. The on-premises MySQL database instead uses a **Self-Hosted Integration Runtime (SHIR)** — a lightweight agent installed on a machine with network access to that database, letting ADF reach it securely without exposing it to the public internet.

How it's used: Azure IR powers cloud-to-cloud movement; Self-Hosted IR is what makes the on-prem MySQL connection possible at all.

- Azure Integration Runtime — Blob Storage & ADLS Gen2
- Self-Hosted Integration Runtime (SHIR) — on-prem MySQL

Pipelines & Data Flows

Step 7 — Building Pipelines & Data Flows per Layer



The Factory Resources panel shows the real shape of the build: 5 pipelines and 8 data flows. Each medallion stage — and each source dataset — gets its own dedicated pipeline or data flow rather than one monolithic process, so any single stage can be fixed or re-run without touching the others.

How it's used: Mapping Data Flows give visual, Spark-powered ETL transformations without writing Spark code by hand.

- Pipelines: On_PremtoCloud_pl, rawtobronze_pl, bronzetosilver_pl, SilverttoGold_pl, Apollo_pl (master)
- Data flows: Apollo_DF plus one per entity — Patient, Provider, PCP, Encounter, Diagnosis, Pregnancy, Location

Bronze Layer

Step 8 — Landing Raw Data in Bronze

The screenshot shows the Azure Storage browser interface for the 'apolloadlstorage' storage account. The breadcrumb path is 'Home > Resource Manager | Resource groups > Apollo_RS > apolloadlstorage'. The interface displays the 'apollo' container, which contains a 'bronze' sub-container. The authentication method is 'Access key (Switch to Microsoft Entra user account)'. A search bar is present with the text 'Search blobs by prefix (case-sensitive)'. Below the search bar, a table lists 7 items:

Name	Last modified	Access tier	Blob type	Size	Lease state
[.]					...
Encounter_Table.xlsx	7/25/2026, 11:37:08 AM	Hot (Inferred)	Block blob	5.2 MiB	Available
Encounter_dx_Table.csv	7/25/2026, 11:37:08 AM	Hot (Inferred)	Block blob	6.45 MiB	Available
Location_Table.csv	7/25/2026, 11:37:08 AM	Hot (Inferred)	Block blob	234 B	Available
OnpremData.csv	7/25/2026, 11:37:08 AM	Hot (Inferred)	Block blob	1.37 MiB	Available
PCP_Table.csv	7/25/2026, 11:37:08 AM	Hot (Inferred)	Block blob	468.85 KiB	Available
Pregnancy_Diagnosis.csv	7/25/2026, 11:37:08 AM	Hot (Inferred)	Block blob	15.35 KiB	Available
Provider.json	7/25/2026, 11:37:08 AM	Hot (Inferred)	Block blob	6.36 KiB	Available

All three sources — Blob Storage, ADLS Gen2, and the on-prem MySQL database — land here, in the Bronze layer, in their raw / near-original form. Nothing is transformed yet — Bronze is the immutable 'source of truth' copy that allows reprocessing at any time without going back to the original source systems.

How it's used: Acts as the single raw staging zone before any cleansing begins.

Silver Layer

Step 9 — Cleansing Data into Silver

The screenshot shows the Azure Storage browser interface for the 'apolloadlstorage' storage account. The breadcrumb path is 'Home > Resource Manager | Resource groups > Apollo_RS > apolloadlstorage'. The interface displays the 'silver' container under the 'medal' container. The authentication method is 'Access key'. A search bar is present with the text 'Search blobs by prefix (case-sensitive)'. A table lists 7 items, all of which are CSV files. The table columns are: Name, Last modified, Access tier, Blob type, Size, and Lease state. The items are: '[.]', 'Encounter_Table_S.csv', 'Encounter_dx_Table_S.csv', 'Location_Table_S.csv', 'PCP_Table_S.csv', 'Pregnancy_Diagnosis_S.csv', and 'Provider_S.csv'. All items have an 'Access tier' of 'Hot (Inferred)', a 'Blob type' of 'Block blob', and a 'Lease state' of 'Available'.

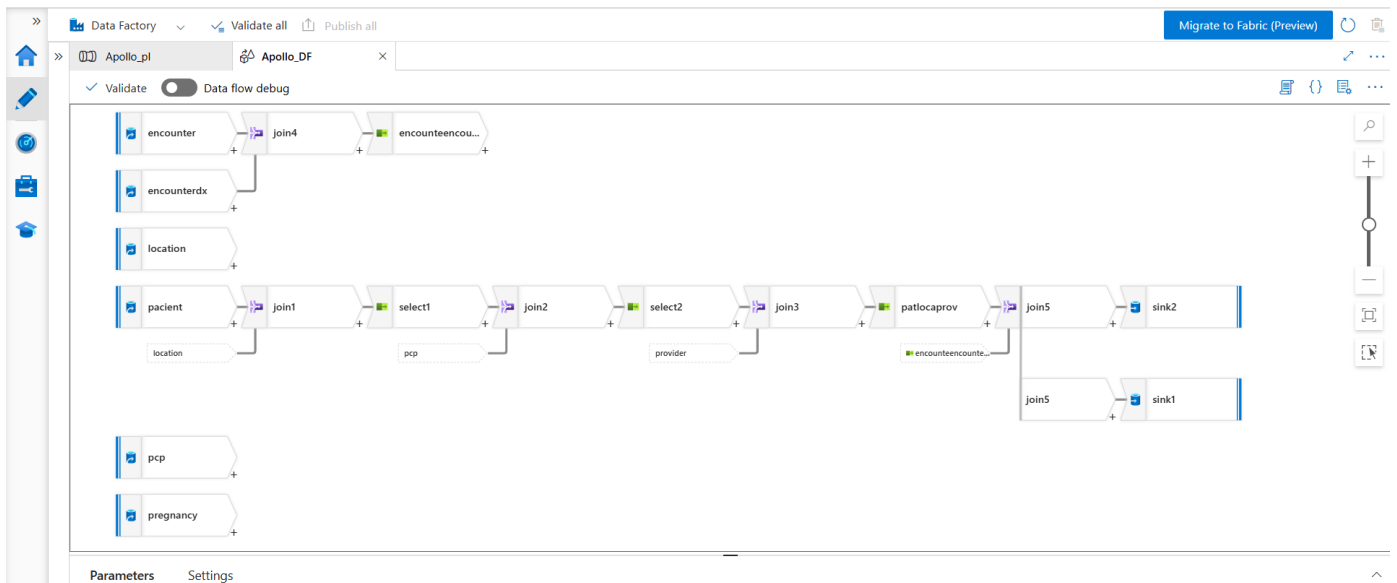
Name	Last modified	Access tier	Blob type	Size	Lease state
[.]					
Encounter_Table_S.csv	7/25/2026, 12:06:26 PM	Hot (Inferred)	Block blob	5.58 MiB	Available
Encounter_dx_Table_S.csv	7/25/2026, 12:06:09 PM	Hot (Inferred)	Block blob	6.07 MiB	Available
Location_Table_S.csv	7/25/2026, 12:05:40 PM	Hot (Inferred)	Block blob	175 B	Available
PCP_Table_S.csv	7/25/2026, 12:05:47 PM	Hot (Inferred)	Block blob	420.02 KiB	Available
Pregnancy_Diagnosis_S.csv	7/25/2026, 12:05:47 PM	Hot (Inferred)	Block blob	20.88 KiB	Available
Provider_S.csv	7/25/2026, 12:05:38 PM	Hot (Inferred)	Block blob	1.69 KiB	Available
patient_table_S.csv	7/25/2026, 12:06:28 PM	Hot (Inferred)	Block blob	969.38 KiB	Available

The Silver data flow removes bad and duplicate records and standardizes the data before writing it here. Silver is the 'trusted' layer — the version of the data that's reliable enough for further joins and business logic to build on.

How it's used: This is the layer downstream jobs and analysts can actually depend on.

Gold Data Flow

Step 10 — The Final Data Flow (Apollo_DF)



This is **Apollo_DF**, the final data flow in the chain. It brings together the individually cleansed Patient, Provider, PCP, Encounter, Diagnosis, Pregnancy, and Location data flows, joining and aggregating them into the finished, business-ready output — writing the result into the Gold layer and Azure SQL Database in the same run.

How it's used: This is where seven separate, cleaned entity datasets finally become one connected, analysis-ready output.

Gold Layer

Step 11 — Business-Ready Data in Gold

The screenshot displays the Azure Storage browser interface for the 'apolloadlstorage' storage account. The left sidebar shows the navigation pane with 'Application Templates' selected. The main pane shows the 'Blob containers' view, specifically the 'apollo' container, which contains a 'medal' folder and a 'gold' folder. The 'gold' folder is selected, showing a table of blobs. The table has columns: Name, Last modified, Access tier, Blob type, Size, and Lease state. One item is listed: 'Apollo_G.csv', which is a 'Block blob' of size '13.35 MiB', last modified on '7/25/2026, 1:30:57 PM', with an 'Access tier' of 'Hot (Inferred)' and a 'Lease state' of 'Available'. The interface also includes a search bar and a filter for 'Only show active objects'.

Name	Last modified	Access tier	Blob type	Size	Lease state
Apollo_G.csv	7/25/2026, 1:30:57 PM	Hot (Inferred)	Block blob	13.35 MiB	Available

The final, joined and aggregated dataset lands here — the Gold layer. This is the business-ready output of the whole pipeline, structured for direct consumption by reporting or analytics tools.

How it's used: The last stop in the data lake before the data is also pushed to a relational database.

Azure SQL Database

Step 12 — Serving the Final Data in Azure SQL Database

Home > Resource Manager | Resource groups > Apollo_RS > Apollo_db (apolloproject/Apollo_db)

Apollo_db (apolloproject/Apollo_db) | Query editor (preview)

SQL database

New query

Templates

Open in

Feedback

Sign out

Classic experience

Explorer

Apollo_db

dbo

Tables

Aopollo_tb

Columns

Indexes

Views

Stored Procedures

Functions

Queries

Data preview - Aopollo_tb

Showing 1000 rows

Search

	128	unique_p...	ABC gender	ABC loc_name	ABC loc_sa	128	pat_pcp_id	128	Age	128	primary...	ABC primary...	ABC primary...	123	enc_id	123	enc_patie
1	463		Male	Location5	MD	35		108		50	Kara Hohlt	Internal Medi...		6384		463	
2	463		Male	Location5	MD	35		108		50	Kara Hohlt	Internal Medi...		6384		463	
3	471		Male	Location5	MD	14		41		31	Oliver Sutterfi...	Family Practice		35100		471	
4	471		Male	Location5	MD	14		41		31	Oliver Sutterfi...	Family Practice		107555		471	
5	496		Female	Location5	MD	5		41		15	Kara Lamantia	Internal Medi...		27792		496	
6	496		Female	Location5	MD	5		41		15	Kara Lamantia	Internal Medi...		27792		496	
7	496		Female	Location5	MD	5		41		15	Kara Lamantia	Internal Medi...		27792		496	
8	496		Female	Location5	MD	5		41		15	Kara Lamantia	Internal Medi...		27792		496	
9	496		Female	Location5	MD	5		41		15	Kara Lamantia	Internal Medi...		27792		496	
10	496		Female	Location5	MD	5		41		15	Kara Lamantia	Internal Medi...		33926		496	
11	496		Female	Location5	MD	5		41		15	Kara Lamantia	Internal Medi...		80041		496	
12	833		Male	Location1	MD	36		40		44	Kara Lamantia	Family Practice		16207		833	
13	833		Male	Location1	MD	36		40		44	Kara Lamantia	Family Practice		69587		833	
14	833		Male	Location1	MD	36		40		44	Kara Lamantia	Family Practice		97408		833	
15	833		Male	Location1	MD	36		40		44	Kara Lamantia	Family Practice		146037		833	

Succeeded (0 sec 483 ms)

Columns: 17 Rows: 1000

Alongside the Gold layer in the data lake, the same finished dataset is written into Azure SQL Database — giving the project both a data-lake copy and a relational copy, so it's ready for whichever downstream tool needs it (BI tools, ad-hoc SQL queries, applications, etc.).

How it's used: Validated by querying the tables directly and confirming the pipeline's final output.

- Finished data available in two forms: Gold-layer files (ADLS) and Azure SQL Database tables

Pipelines & Data Flows Explained

A pipeline is the orchestration wrapper (it decides when/how something runs); a data flow is the actual visual, Spark-powered transformation logic inside it. Apollo uses 5 pipelines and 8 data flows in total — one pair of ingestion pipelines feeding Bronze, one pipeline running the Silver cleansing flows, one pipeline running the Gold aggregation flow, and a master pipeline tying it all together.

Pipelines

On_PremtoCloud_pl — Copies data from the on-premises MySQL database into Azure storage via the Self-Hosted Integration Runtime — the first hop that gets on-prem data into the cloud before it can join the rest of the flow.

rawtobronze_pl — Ingests raw files from Blob Storage and ADLS Gen2 (alongside the staged MySQL data) into the Bronze layer, unchanged.

bronzetosilver_pl — Runs the per-entity cleansing data flows and writes their standardized output into the Silver layer.

SilvertoGold_pl — Runs the final aggregation data flow (Apollo_DF) and writes its output to both the Gold layer and Azure SQL Database.

Apollo_pl — The master pipeline — runs the four pipelines above in sequence via Execute Pipeline activities, turning the whole project into one schedulable, end-to-end workflow.

Data Flows

Six data flows each clean one healthcare-domain entity independently, and a seventh (Apollo_DF) joins them all together:

patient_table_df — Cleanses and standardizes core Patient records.

Provider_df — Cleanses and standardizes healthcare Provider records.

PCP_Table_df — Cleanses Primary Care Provider (PCP) reference data.

Encounter_dx_df — Cleanses patient Encounter and diagnosis-code data.

Pregnancy_Diagnosis_df — Cleanses Pregnancy-related diagnosis records.

Location_Table_df — Cleanses Location reference data used across the other entities.

Apollo_DF — The final data flow — joins all six cleansed entity data flows above and aggregates them into the finished Gold output, written to both ADLS Gold and Azure SQL Database.

Together, these pipelines and data flows form one continuous chain — on-prem MySQL and cloud files feed Bronze, Bronze feeds the per-entity Silver cleansing, and Silver feeds the single Gold join — so the entire project can be re-run end-to-end from one trigger on Apollo_pl, with every stage remaining independently debuggable.

Azure Services Used — Quick Reference

Service	What it is	How it was used here
Resource Group	Logical container for every resource in the project	Organizes storage, ADF, and Azure SQL Database as one manageable unit (apollo_RS)
Blob Storage	General-purpose object storage	One of three raw source systems feeding the pipeline
ADLS Gen2	Hierarchical, big-data-optimized storage	Second source system + hosts the Bronze / Silver / Gold layers
MySQL	On-premises relational database	Third source system — reached securely from ADF via a Self-Hosted Integration Runtime
Azure Data Factory	Cloud ETL/orchestration service	Linked Services, Integration Runtimes, Pipelines & Data Flows for the whole pipeline
Integration Runtime	Compute that executes ADF activities	Azure IR for Blob/ADLS; Self-Hosted IR (SHIR) for the on-prem MySQL connection
Mapping Data Flows	Visual, Spark-based transformation designer	Cleansing (Bronze→Silver) and joins/aggregation (Silver→Gold)
Azure SQL Database	Managed relational database	Serving layer for the finished Gold data, consumable by BI tools